

Construindo o Harness de IA

Controle e Reprodutibilidade para Agentes Autônomos em Ciência de Dados

Ricardo Ceneviva (UFABC) & Anderson Borba (Mackenzie)

Rede Brasileira de Reprodutibilidade (RBR)

7 de outubro de 2026

Pergunta central da aula

Pergunta-guia

Como permitir que um agente trabalhe sem perder controle sobre suas ações?

- 1 Reconhecer os arquivos de um harness mínimo.
- 2 Distinguir instruções de código de orquestração.
- 3 Localizar permissões, aprovação, rastro e validação no fluxo.
- 4 Interpretar replay como substituto controlado de nova chamada.

Coding agents já escrevem e executam código, coletam dados e produzem resultados. Como controlar e auditar essas ações? *Fonte: Sampaio e Galdino, SciELO em Perspectiva (2026).*

Um caso único: dados regionais de chuva

- Arquivo mensal: Regiões Geográficas Imediatas do Rio Grande do Sul, 2018–2025.
- Chave esperada: `codigo_rgi + mes`.
- Medida: precipitação em milímetros, derivada do CHIRPS v3.0.
- A cópia do exercício pode conter valores vazios e chaves repetidas.

Tarefa no laboratório

Inspecionar a execução e validar os dados antes de aceitar um resultado.

Fonte: CHC/CHIRPS v3.0; IBGE; dicionário dos dados da oficina.

Chatbot, workflow, agente e replay

Forma	Quem define o próximo passo?
Chatbot	A interação segue a conversa; pode ou não usar ferramentas.
Workflow	O caminho principal está definido previamente em código.
Agente	O modelo escolhe ações dentro de limites e recebe retorno do ambiente.
Replay	Uma sequência de ações preparada é reproduzida.

Nesta oficina

O replay fixa as ações preparadas; os controles e validadores são executados de fato.

Fonte: Anthropic, Building Effective Agents; Plano Operacional da oficina.

Da resposta à ação supervisionada

Pedido $\rightarrow$ ação proposta $\rightarrow$ controle $\rightarrow$ ferramenta

- O agente escolhe o próximo passo, como `ler_dados(path)`.
- O harness decide se a ação pode prosseguir e se deve pausar.
- A proposta do modelo **não autoriza** a execução.

Autonomia supervisionada

Há liberdade para propor ações dentro de ferramentas e limites definidos por pessoas.

Anatomia de um harness mínimo

```
harness_demo/  
|-- AGENTS.md  
|-- workflow.md  
|-- harness.py  
|-- replay.json  
|-- participantes/  
    |-- dados/  
        |-- chuva_rs_2018_2025_exercicio.csv  
    |-- saida/
```

- AGENTS.md: instruções.
- workflow.md: mapa do processo.
- harness.py: executa controles.
- replay.json: ações fixas.

Dois planos

Arquivos de instrução documentam regras; harness.py aplica regras na execução.

Árvore ilustrativa; a lógica correspondente está no notebook do laboratório.

AGENTS.md: instruções, não execução

```
# AGENTS.md
Objetivo: analisar a chuva do exercicio.
Ferramentas: ler_dados, gravar_saida.
Regras:
- ler somente participantes/dados/
- gravar somente participantes/saida/
- nunca acessar docente/
- nao executar codigo arbitrario
- pedir aprovacao antes de gravar
- registrar acoes e verificacoes
```

AGENTS.md fornece instruções persistentes no repositório. Codex pode descobri-lo na hierarquia de diretórios; Python customizado deve carregá-lo ou definir `Agent.instructions`. *Fonte: OpenAI, documentação oficial de AGENTS.md e Agents SDK.*

workflow.md: mapa humano do fluxo

```
# workflow.md
1. Receber acao proposta
2. Conferir ferramenta e argumentos
3. Aplicar permissao
4. Pausar para aprovacao, se exigida
5. Executar a ferramenta
6. Validar dados e/ou resultado
7. Registrar evento e evidencia
8. Retornar resultado e evidencia
```

Fronteira

workflow.md documenta o processo; ele não executa o processo sozinho.

harness.py: ferramentas e argumentos

```
TOOLS = {"ler_dados": (ler_dados, {"path"}),
         "gravar_saida": (gravar_saida,
                          {"path", "conteudo"})}

def selecionar(acao):
    nome, args = acao["tool"], acao["args"]
    if nome not in TOOLS or not isinstance(args, dict):
        raise ValueError("acao invalida")
    funcao, campos = TOOLS[nome]
    if set(args) != campos:
        raise ValueError("argumentos invalidos")
    return funcao
```

Só ferramentas em TOOLS são expostas e executáveis. Checks de dados e resultados são funções internas obrigatórias do harness; o modelo não precisa solicitá-los.

harness.py: política de caminhos

```
from pathlib import Path
BASES = {"ler_dados": "participantes/dados",
        "gravar_saida": "participantes/saida"}
def validar_permissao(acao):
    nome = acao["tool"]
    if nome not in BASES: return
    caminho = acao["args"]["path"]
    if not isinstance(caminho, str):
        raise TypeError("path deve ser texto")
    base = Path(BASES[nome]).resolve()
    if not Path(caminho).resolve().is_relative_to(base):
        raise PermissionError("caminho bloqueado")
```

A allowlist bloqueia docente/; é política **do nosso harness**, não recurso automático do Agents SDK.

harness.py: pausa antes do efeito

```
def executar(acao):  
    ferramenta = selecionar(acao)  
    validar_permissao(acao)  
    if acao["tool"] == "gravar_saida":  
        if not pedir_aprovacao(acao):  
            registrar_bloqueio(acao)  
            return None  
    resultado = ferramenta(**acao["args"])  
    evidencia = validar_saida(acao, resultado)  
    registrar_evento(acao, resultado, evidencia)  
    return resultado, evidencia
```

A pessoa aprova ferramenta, argumentos e destino **concretos**. Se mudarem, há nova ação. Recusa não chama a ferramenta. Após executar: resultado → evidência → trace.

Onde cada controle atua?

- 1 **Instruções** orientam a proposta de ação.
- 2 **Registry + argumentos** aceitam ferramenta e formato conhecidos.
- 3 **Permissão** bloqueia caminho ou ação proibida.
- 4 **Aprovação** pausa uma ação sensível e pode recusá-la.
- 5 **Ferramenta** executa a ação aceita.
- 6 **Validadores internos** calculam evidência sobre dados e resultado.
- 7 **Trace** registra resultado e evidência; o **retorno** os entrega.

Dois planos conectados

Arquivo de instruções não é código de orquestração. O `harness.py` aplica a sequência.

Trace: evento observável, sem raciocínio interno

```
{  
  "tool": "ler_dados",  
  "permission": "allowed",  
  "status": "executed",  
  "validation": "checked"  
}
```

Trace operacional: o que o sistema fez. **Proveniência científica:**

ferramenta/modelo/versão, data, dados, configuração e artefatos produzidos.

O SDK rastreia eventos próprios; o harness deve anexar checks e evidências calculadas. Trace operacional não substitui proveniência científica. *Fonte: OpenAI, Agents SDK: tracing; Sampaio, Sabbatini e Limongi (2024).*

Validar dados: calcular evidência

```
def checar_dados(df):  
    return {  
        "duplicadas": int(  
            df.duplicated(["codigo_rgi", "mes"]).sum()  
        ),  
        "ausentes": int(  
            df["precipitacao_mm"].isna().sum()  
        ),  
    }
```

checar_dados é função interna chamada por validar_saida após ler_dados. Os números calculados formam evidência anexável ao trace. O CSV inicial **contém falhas didáticas**; a oficina localizará as ocorrências. *Fonte: Dicionário do exercício e validação da Fase 2.*

Validar resultado: calcular e comparar

```
def conferir_media(df_limpo, media_reportada):  
    esperado = df_limpo["precipitacao_mm"].mean()  
    erro = abs(media_reportada - esperado)  
    return {"media_recalculada": esperado,  
            "erro_absoluto": erro,  
            "confere": erro < 1e-6}
```

Interpretação

O valor reportado só confere se reproduzir a mesma regra, período e seleção de linhas. Imprimir OK não verifica nada.

`conferir_media` é função interna para o resultado analítico pertinente; `validar_saida` devolve a comparação, e `registrar_evento` a anexa ao trace. O modelo não precisa solicitar esse check.

replay.json: ações preparadas

```
[
  {
    "tool": "ler_dados",
    "args": {
      "path": "participantes/dados/chuva_rs_2018_2025_exercicio.csv"
    }
  }
]
```

O replay **fixa as ações propostas** e substitui a nova chamada ao modelo. A validação interna roda automaticamente após ler_dados; permissões e trace também executam de fato. *Fonte: Estrutura ilustrativa; o replay operacional está no laboratório.*

Replay não é uma nova chamada ao modelo

Replay

Mesmas ações preparadas permitem testar o harness sob uma sequência conhecida.

- Mesmos dados e código ajudam a repetir cálculos.
- Eles não garantem identidade entre chamadas novas ao modelo.

Nova chamada

O modelo pode produzir outra resposta ou escolher outras ações.

Fonte: OpenAI, documentação da API sobre determinismo de saídas.

Reprodutibilidade e validade científica

- Reprodutibilidade computacional: obter resultados consistentes com os mesmos dados, passos, código e condições.
- Em IA, condições observáveis podem incluir modelo/versão, data, configuração, prompts/instruções e artefatos.
- Replay fixa ações conhecidas; registrar essas condições melhora a proveniência, mas não garante saídas idênticas em novas chamadas.
- Validade científica exige também examinar fonte, medida, método e interpretação.
- Um cálculo reproduzível pode continuar errado ou inadequado à pergunta.

Limite do harness

Ele cria condições para controle e inspeção; **não garante correção científica.**

Fonte: National Academies (2019); README da oficina.

Síntese

IA + harness → autonomia supervisionada + registros + controles + validação → melhores condições de qualidade e eficiência.

- Localizar no `harness.py` onde cada ação é permitida, pausada ou bloqueada.
- No laboratório: completar **um** controle e testar os **três**.
- Explicar o trace, os checks de dados e a comparação do resultado.

Referências

ANTHROPIC. *Building Effective Agents*. 2024.

OPENAI. *Harness engineering*. 2026.

OPENAI. *Custom instructions with AGENTS.md*.
Documentação do Codex.

OPENAI. Agents SDK: instruções e ferramentas,
aprovação e tracing.

SAMPAIO; GALDINO. *Algo grande está
acontecendo....* SciELO, 2026.

OPENAI. *Text generation*. Documentação da API.

NATIONAL ACADEMIES. *Reproducibility and
Replicability in Science*. 2019.

CLIMATE HAZARDS CENTER. *CHIRPS v3.0:
README*.

IBGE. *Regiões Geográficas Imediatas: malha
2017*.

SAMPAIO; SABBATINI; LIMONGI. *Diretrizes
para o uso ético de IA*. Intercom, 2024.